

CHAPTER 3: ANALYSIS

3.1 Introduction

This chapter presents the requirement analysis phase for FOODIE-HALAL-SNAP. The analysis phase is concerned with thoroughly investigating the current situation and problems faced by the target users and translating these findings into precise system requirements that will guide the design and implementation of the application. The chapter is organized as follows: Section 3.2 analyses the current system scenario and the problems that the proposed system must address. Section 3.3 defines the detailed system requirements across four dimensions — data, functional, non-functional, and additional technical requirements. Section 3.4 summarizes the chapter and previews the next development activities.

3.2 Problem Analysis

One of the existing applications used for halal verification is the eHalal: Halal Scanner & Map application. This application allows users to scan the barcode of packaged food products to check their halal certification status using a JAKIM-linked database. It also provides features such as locating nearby halal-certified restaurants.

However, the functionality of eHalal is limited to packaged food items that contain barcodes. It cannot be used for prepared dishes such as meals served in restaurants or homemade food, where no barcode is available. As a result, users are

unable to verify the halal status of such dishes using this application, making it unsuitable for real-world scenarios involving unfamiliar prepared food.

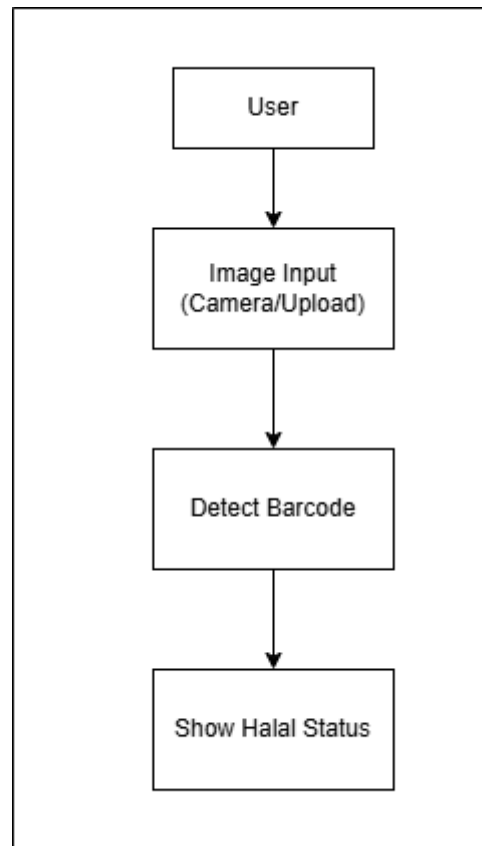


Figure 3.1: System Architecture Diagram

As illustrated in the activity diagram, the halal verification process using the eHalal application begins when the user provides an image input through camera capture or upload. The system then attempts to detect a barcode from the input. If a barcode is successfully detected, the application proceeds to display the halal status of the product.

However, this process is highly dependent on the presence of a barcode. For most prepared dishes, such as meals served in restaurants or homemade food, no barcode is available. As a result, the barcode detection step fails, preventing the system from proceeding to halal verification. This limitation demonstrates that applications such as eHalal are not suitable for verifying the halal status of unpackaged or prepared dishes, leading to unreliable and incomplete verification for users.

The three core problems identified from this analysis are:

- Problem 1 — Unknown dish identification: Users cannot identify the name or ingredients of an unfamiliar Malaysian dish based solely on visual observation. This limitation prevents users from performing effective text-based searches for recipes or additional information, as most search platforms require prior knowledge of the dish name.
- Problem 2 — Unreliable halal verification for prepared dishes: Existing tools such as the eHalal application rely on barcode scanning to verify halal certification. However, this approach is not applicable to prepared dishes that do not contain barcodes, such as meals served in restaurants or homemade food. As a result, users must depend on informal sources such as asking staff or making assumptions, which may lead to uncertainty and unreliable halal verification.
- Problem 3 — Fragmented multi-step user journey: The current process requires users to switch between multiple platforms, including search engines for dish identification, applications such as eHalal for halal verification, and recipe websites for cooking instructions. This fragmented workflow is inefficient, time-consuming, and inconvenient for users who need quick and reliable information.

These problems define the system scenario that FOODIE-HALAL-SNAP aims to solve. In the proposed system, users capture or upload an image of a Malaysian dish, and the system automatically identifies the dish, determines its halal status using a JAKIM-compliant database, and retrieves the corresponding recipe within a single application.

Each problem is addressed by a specific system module. Problem 1 is handled by the Food Dish Detection Module using a YOLOv8 model. Problem 2 is solved by the Halal Status Classification Module through a rule-based database lookup. Problem 3 is addressed by the integration of the Recipe Retrieval Module and the Mobile User Interface Module, providing a seamless user experience.

3.3 Requirement Analysis

3.3.1 Data Requirements

The system processes food images as the primary input. Users interact with the system through two input mechanisms: live camera capture, where the user photographs a dish using the device camera, and gallery upload, where the user selects an existing image from the device photo library. The system output is presented on a result screen displaying the detected dish name, halal classification status with a color-coded indicator, and a navigation link to the recipe detail screen.

The data stored within the system consists of one local database for halal classification and an external API for recipe retrieval. The recipe data is dynamically fetched from TheMealDB API at runtime. Table 3.1 presents the data dictionary for the system.

Table 3.1: Data Dictionary

Field Name	Data Type	Source	Location	Description
dish_label	String	YOLOv8 model output	Runtime	The detected Malaysian dish category name (e.g., 'Nasi Lemak', 'Laksa') output by the food detection model.
confidence_score	Float (0–1)	YOLOv8 model output	Runtime	The model prediction confidence for the detected dish label. Detections below 0.5 threshold prompt the user to retake or upload a clearer image.
halal_status	String (enum)	Halal DB lookup	Local JSON DB	The halal classification of the detected dish: 'Halal', 'Haram', or 'Syubhah',

Field Name	Data Type	Source	Location	Description
				assigned based on JAKIM guidelines and MS 1500:2019.
ingredients	Array of String	TheMealDB API	Runtime	Retrieved dynamically from API response
instructions	String	TheMealDB API	Runtime	Retrieved dynamically from API response

The data model for FOODIE-HALAL-SNAP is straightforward. The input image is processed by the TFLite model at runtime to produce a dish_label and confidence_score. The dish_label serves as the foreign key for one local database lookup (halal classification) and one external API call (recipe retrieval via TheMealDB API).

3.3.2 Functional Requirements

The functional requirements specify the functions that FOODIE-HALAL-SNAP must perform. Figure 3.2 presents the Use Case Diagram illustrating the interactions between the system actors and the system functions.

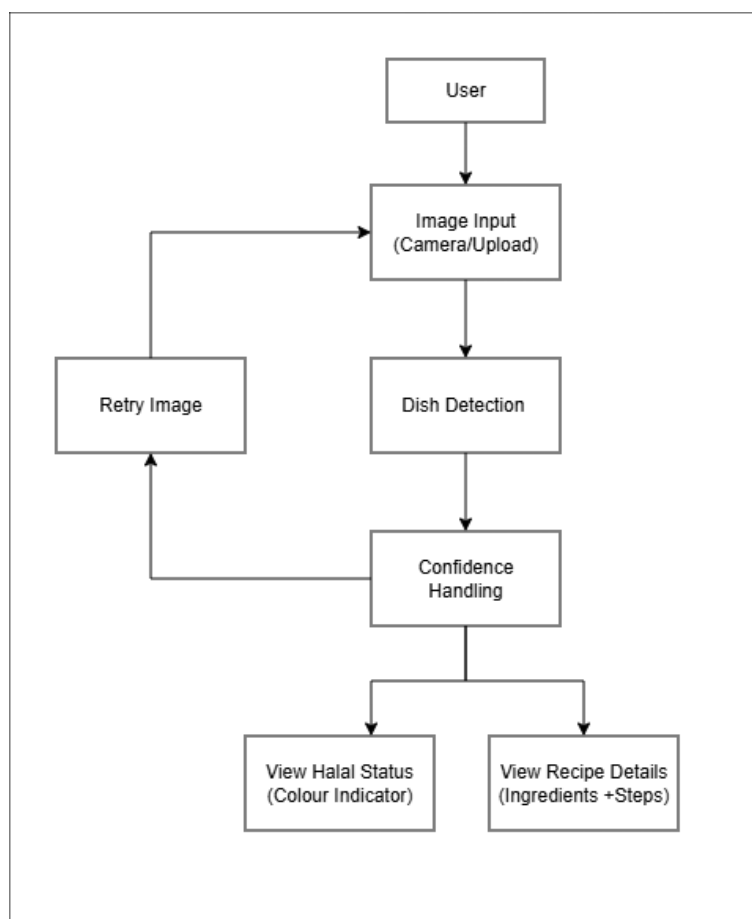


Figure 3.2: Use Case Diagram for FOODIE-HALAL-SNAP

The system functions are specified as follows:

- FR1 — Image Input: The system shall accept food image input via live camera capture or device gallery upload.
- FR2 — Dish Detection: The system shall process the input image through the on-device YOLOv8 TFLite model and return a dish_label and confidence_score for the detected Malaysian dish.

- FR3 — Confidence Handling: If the confidence_score is below 0.5, the system shall display a prompt asking the user to retake or upload a clearer image.
- FR4 — Halal Status Indicator: The system shall display the halal_status with a colour-coded visual indicator: green for Halal, red for Haram, and yellow for Syubhah.
- FR5 — Recipe Retrieval: The system shall retrieve and display the ingredients and instructions for the detected dish_label by sending a request to TheMealDB API and processing the returned response.
- FR6 — API Request Handling: The system shall send HTTP requests to TheMealDB API and parse JSON responses to extract recipe details.

The functional pipeline operates as follows. The user launches the application and is presented with the home screen. The user selects camera capture or gallery upload (FR1). The captured image is passed to the TFLite inference engine, which returns a dish_label and confidence_score (FR2). If the confidence is below threshold, the user is prompted to retry (FR3). Otherwise, the result screen is displayed showing the dish name, halal status with colour-coded indicator (FR4), and retrieved dynamically from TheMealDB API (FR5, FR6).

3.3.3 Non-Functional Requirements

The non-functional requirements specify how well the system performs its intended functions across the following quality dimensions:

Performance

- The system shall deliver end-to-end detection results within a maximum median response time of three seconds from image submission on a mid-range Android (minimum Snapdragon 665) device.
- The food detection model shall achieve a minimum mAP@0.5 of 80% across all ten supported dish categories on the held-out test set.
- Recipe retrieval response from TheMealDB API shall be completed within 2 seconds under normal internet conditions.

Reliability

- The application shall operate without crashes or significant performance degradation during standard usage sessions on target mobile hardware.
- Core functions shall remain available without internet connectivity through on-device TFLite inference and local JSON databases.
- The system shall handle API failures gracefully by displaying an error message or fallback response when recipe data cannot be retrieved.

Usability

- The application shall achieve a minimum System Usability Scale (SUS) score of 70, corresponding to the 'Good' usability grade, assessed with a minimum of five representative target users.
- Core functions (image capture, result viewing, recipe navigation) shall be accessible within two taps from the application home screen.

Compatibility

- The application shall be compatible with Android devices running Android 8.0 (API level 26).

Data Storage

- The total application package size including the TFLite model and local halal database and external API-based recipe retrieval shall not exceed 100 MB to remain within acceptable download size limits for consumer mobile applications.

3.3.4 Other Requirement

The following additional software and hardware requirements are specified for the development and deployment of FOODIE-HALAL-SNAP, with justification for each.

Python 3.10 and the Ultralytics YOLOv8 library are required for model training and TFLite conversion. Python is the de facto standard language for deep learning research and provides the broadest compatibility with available machine learning libraries including PyTorch, which is the underlying framework for YOLOv8 training (Jocher et al., 2023). A CUDA-capable NVIDIA GPU with a minimum of 6 GB VRAM is required for YOLOv8 training with acceptable speed; CPU-only training is possible but would require prohibitively long training times for a 10-class 640x640-input model.

Flutter SDK 3.0 or above and the `tflite_flutter` plugin are required for mobile application development. Flutter is selected over native development because it enables a single codebase to compile to both Android, reducing development time while maintaining native performance. The `tflite_flutter` plugin provides the bridge between the Flutter application and the on-device TFLite model, and its compatibility with TFLite 2.x is verified for the project's deployment target.

Physical Android test devices are required for performance and usability testing. Emulators and simulators cannot accurately replicate the camera hardware behaviour, real-device inference performance, or touch interaction characteristics needed for valid application testing. Minimum device specifications are Snapdragon 665 for Android, representing mid-range consumer hardware that reflects the realistic target user population.

3.4 Summary

This chapter has presented the requirement analysis for FOODIE-HALAL-SNAP. Section 3.2 analyzed the current system scenario through an activity diagram of the manual food identification and halal verification process, identifying three core problems: unknown dish identification, unreliable halal verification for prepared dishes, and a fragmented multi-step user journey. Section 3.3 defined the complete system requirements across four dimensions. The data requirements in Section 3.3.1 specified the input mechanisms, output displays, and data dictionary covering five data fields managed through two local JSON databases. The functional requirements in Section 3.3.2 defined eight system functions (FR1–FR8) supported by a Use Case Diagram. The non-functional requirements in Section 3.3.3 specified quantitative performance, reliability, usability, compatibility, and storage criteria. Section 3.3.4 provided justified additional hardware and software requirements.